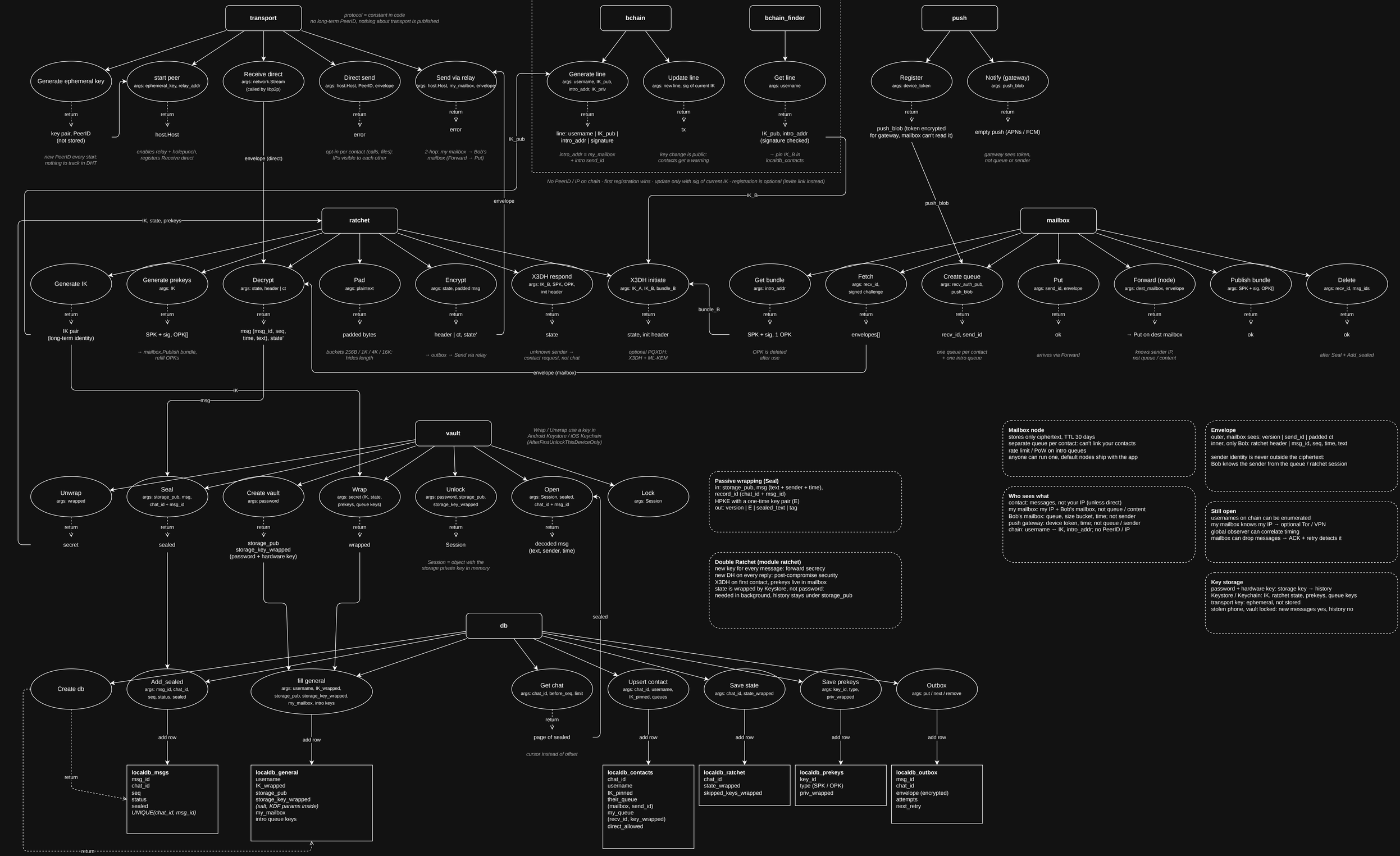

transport
protocol = constant in code
no long-term PeerID, nothing about transport is published
Generate ephemeral key
return
key pair, PeerID
(not stored)
new PeerID every start:
nothing to track in DHT
start peer
args: ephemeral_key, relay_addr
return
host.Host
enables relay + holepunch,
registers Receive direct
Receive direct
args: network.Stream
(called by libp2p)
envelope (direct)
Direct send
args: host.Host, PeerID, envelope
return
error
opt-in per contact (calls, files):
IPs visible to each other
Send via relay
args: host.Host, my_mailbox, envelope
return
error
2-hop: my mailbox → Bob's
mailbox (Forward → Put)
IK_pub
envelope
bchain
Generate line
args: username, IK_pub,
intro_addr, IK_priv
return
line: username | IK_pub |
intro_addr | signature
intro_addr = my_mailbox
+ intro send_id
Update line
args: new line, sig of current IK
return
tx
key change is public:
contacts get a warning
bchain_finder
Get line
args: username
return
IK_pub, intro_addr
(signature checked)
→ pin IK_B in
localdb_contacts
No PeerID / IP on chain · first registration wins · update only with sig of current IK · registration is optional (invite link instead)
IK_B
push
Register
args: device_token
return
push_blob (token encrypted
for gateway, mailbox can't read it)
Notify (gateway)
args: push_blob
return
empty push (APNs / FCM)
gateway sees token,
not queue or sender
push_blob
IK, state, prekeys
ratchet
Generate IK
return
IK pair
(long-term identity)
Generate prekeys
args: IK
return
SPK + sig, OPK[]
→ mailbox.Publish bundle,
refill OPKs
Decrypt
args: state, header | ct
return
msg (msg_id, seq,
time, text), state'
Pad
args: plaintext
return
padded bytes
buckets 256B / 1K / 4K / 16K:
hides length
Encrypt
args: state, padded msg
return
header | ct, state'
→ outbox → Send via relay
X3DH respond
args: IK_B, SPK, OPK,
init header
return
state
unknown sender →
contact request, not chat
X3DH initiate
args: IK_A, IK_B, bundle_B
return
state, init header
optional PQXDH:
X3DH + ML-KEM
bundle_B
mailbox
Get bundle
args: intro_addr
return
SPK + sig, 1 OPK
OPK is deleted
after use
Fetch
args: recv_id,
signed challenge
return
envelopes[]
Create queue
args: recv_auth_pub,
push_blob
return
recv_id, send_id
one queue per contact
+ one intro queue
Put
args: send_id, envelope
return
ok
arrives via Forward
Forward (node)
args: dest_mailbox, envelope
return
→ Put on dest mailbox
knows sender IP,
not queue / content
Publish bundle
args: SPK + sig, OPK[]
return
ok
Delete
args: recv_id, msg_ids
return
ok
after Seal + Add_sealed
envelope (mailbox)
IK
msg
vault
Wrap / Unwrap use a key in
Android Keystore / iOS Keychain
(AfterFirstUnlockThisDeviceOnly)
Unwrap
args: wrapped
return
secret
Seal
args: storage_pub, msg,
chat_id + msg_id
return
sealed
Create vault
args: password
return
storage_pub
storage_key_wrapped
(password + hardware key)
Wrap
args: secret (IK, state,
prekeys, queue keys)
return
wrapped
Unlock
args: password, storage_pub,
storage_key_wrapped
return
Session
Session = object with the
storage private key in memory
Open
args: Session, sealed,
chat_id + msg_id
return
decoded msg
(text, sender, time)
Lock
args: Session
Passive wrapping (Seal)
in: storage_pub, msg (text + sender + time),
record_id (chat_id + msg_id)
HPKE with a one-time key pair (E)
out: version | E | sealed_text | tag
Double Ratchet (module ratchet)
new key for every message: forward secrecy
new DH on every reply: post-compromise security
X3DH on first contact, prekeys live in mailbox
state is wrapped by Keystore, not password:
needed in background, history stays under storage_pub
Mailbox node
stores only ciphertext, TTL 30 days
separate queue per contact: can't link your contacts
rate limit / PoW on intro queues
anyone can run one, default nodes ship with the app
Who sees what
contact: messages, not your IP (unless direct)
my mailbox: my IP + Bob's mailbox, not queue / content
Bob's mailbox: queue, size bucket, time; not sender
push gateway: device token, time; not queue / sender
chain: username ↔ IK, intro_addr; no PeerID / IP
Envelope
outer, mailbox sees: version | send_id | padded ct
inner, only Bob: ratchet header | msg_id, seq, time, text
sender identity is never outside the ciphertext:
Bob knows the sender from the queue / ratchet session
Still open
usernames on chain can be enumerated
my mailbox knows my IP → optional Tor / VPN
global observer can correlate timing
mailbox can drop messages → ACK + retry detects it
Key storage
password + hardware key: storage key → history
Keystore / Keychain: IK, ratchet state, prekeys, queue keys
transport key: ephemeral, not stored
stolen phone, vault locked: new messages yes, history no
db
sealed
Create db
return
Add_sealed
args: msg_id, chat_id,
seq, status, sealed
add row
localdb_msgs
msg_id
chat_id
seq
status
sealed
UNIQUE(chat_id, msg_id)
fill general
args: username, IK_wrapped,
storage_pub, storage_key_wrapped,
my_mailbox, intro keys
add row
localdb_general
username
IK_wrapped
storage_pub
storage_key_wrapped
(salt, KDF params inside)
my_mailbox
intro queue keys
Get chat
args: chat_id, before_seq, limit
return
page of sealed
cursor instead of offset
Upsert contact
args: chat_id, username,
IK_pinned, queues
add row
localdb_contacts
chat_id
username
IK_pinned
their_queue
(mailbox, send_id)
my_queue
(recv_id, key_wrapped)
direct_allowed
Save state
args: chat_id, state_wrapped
add row
localdb_ratchet
chat_id
state_wrapped
skipped_keys_wrapped
Save prekeys
args: key_id, type,
priv_wrapped
add row
localdb_prekeys
key_id
type (SPK / OPK)
priv_wrapped
Outbox
args: put / next / remove
add row
localdb_outbox
msg_id
chat_id
envelope (encrypted)
attempts
next_retry
return
First contact (Alice → Bob)
Alice: Get line(bob),
verify sig, pin IK_B
Get bundle from Bob's mailbox,
verify SPK sig with IK_B
X3DH initiate
+ Create queue for Bob
Contact request → Bob's intro
queue (IK_A, EK_A, send_id, hello)
Bob accepts: X3DH respond,
queue for Alice, sends its send_id
Every message
Seal own copy (pending),
Pad + Encrypt
envelope → outbox
Send via relay:
my mailbox → Bob's mailbox
Bob's mailbox → gateway
→ empty push
Bob: Fetch → Decrypt
→ Seal → Add_sealed
Delete + ACK (ratchet msg),
Alice: delivered, clear outbox
Optional direct mode: if both contacts set direct_allowed, envelopes go over libp2p directly (ephemeral PeerIDs exchanged inside the ratchet). Same envelope, faster, but IPs are visible to each other.